

AspireApp1 - Sammanfattning av arbetet

Bygge, lokal körning, felrättningar och ny visualisering · Repo `hkzd8h2sqd-ui/AspireAppExampleApiServices` · 30 september 2026

2

pull requests

43 / 43

tester gröna

0 / 0

varningar / fel

7 / 7

förslag byggda

3

buggar rättade

1. Kort sammanfattning

- Lösningen (.NET 10, Aspire 13.5, Blazor Server, EF Core) **bygger och körs**, både i molncontainern och lokalt på iMac.
- SQLite är nu standard** för StateStore, så att appen går att köra på macOS utan SQL Server.
- Uppsättningen på macOS är dokumenterad steg för steg, inklusive felsökning av HTTPS-certifikatet.
- Tre buggar är rättade. Den viktigaste gjorde att startsida, körningslista och processflöde tyst gav tomma resultat på SQLite.
- En ny sida, **Insikter**, och en förbättrad tidslinje visar trender och mönster över många körningar. Alla sju förslag från analysrapporten är byggda.

2. Hur arbetet gick till

Bygga och köra. Installerade .NET 10 SDK och Aspire CLI i containern, byggde lösningen och startade alla tjänster. Skärmdump tagen med Playwright.

SQLite som standard. SQL Server-standard (`Trusted_Connection`) fungerar inte på macOS. SQLite blev standard i konfiguration, AppHost och `ResolveProvider`.

Lokal körning på iMac. Löste i tur och ordning: saknad SDK-version (10.0.303), `zsh: killed` efter SDK-uppdatering, saknad Aspire CLI och ett HTTPS-certifikat som inte gick att lita på via SSH från iPad (Termius). Certifikatet löstes i Terminal.app vid datorn.

VS Code. Lade till tasks och launch-konfiguration: `⌘⇧B` bygger, `F5` startar AppHost.

Analys och rapport. Gick igenom vad appen visualiserar och vilken data i StateStore som inte används. Resultatet blev sju prioriterade förslag i en PDF-rapport.

Bygga förslagen. Implementerade alla sju, testade mot riktig SQLite och verifierade i den körande appen med riktiga flöden och en avstängd tjänst.

3. Pull requests

PR #1 **bas**

SQLite som standard, guide för lokal körning, VS Code-konfiguration, analysrapporten (PDF) och **SQLite-rättelsen**.

`claude/build-run-locally-wlecgw → master`

PR #2 **bygger på #1**

Insiktssidan, förbättrad tidslinje, rättelse av "Okänd"-status, 20 nya tester, dokumentation och skärmdumpar.

`claude/insights-visualizations`

Ordning: slå ihop PR #1 först och sedan PR #2. GitHub byter automatiskt bas för #2 till `master`.

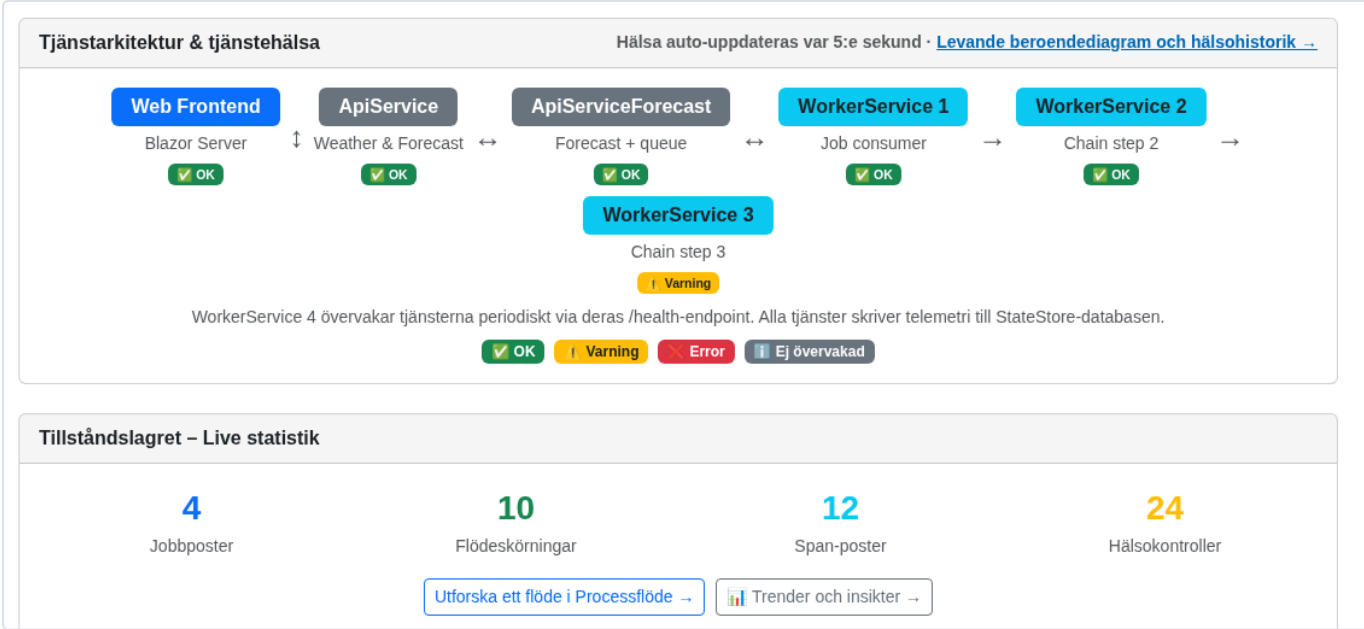
4. Rättade buggar

SQLite kunde inte sortera på datum (PR #1). SQLite kan inte översätta DateTimeOffset i ORDER BY eller WHERE . När SQLite blev standard kastade därför startsidan, "Alla körningar" och alla uppslag i Processflöde undantag som fångades tyst. Resultatet var "Ej övervakad" på alla tjänster och tomma listor.

Åtgärd: sortera på Id i databasen och på datum i minnet (ToListAsyncOrderedByAsync). **Verifierat:** nya tester mot riktig SQLite misslyckades före rättelsen och går igenom efter.

Timeout visades som "Okänd" (PR #2). En körning som fick timeout utan att något steg markerats som misslyckat visades som "Okänd". Nu visas den som "Fel", med steget där den stannade.

Beroendediagram utan kanter. Bara väderflödet skriver spans, så en graf enbart från spans fick en enda nod. Grafen använder nu också flödesstegens överlämningar och visar den verkliga kedjan WorkerService1 → 2 → 3.

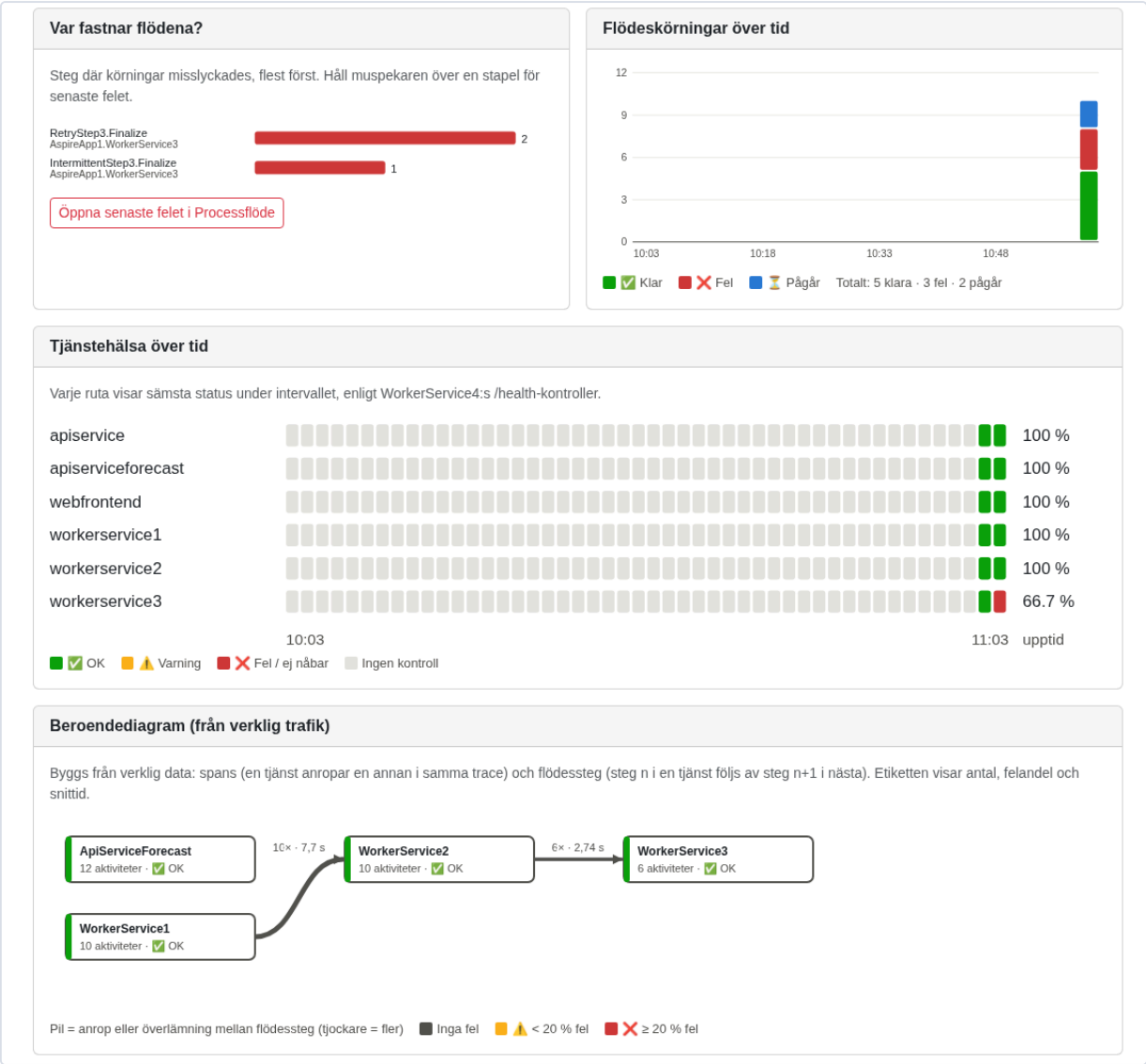


Startsidan efter SQLite-rättelsen: hälsostatus visas igen.

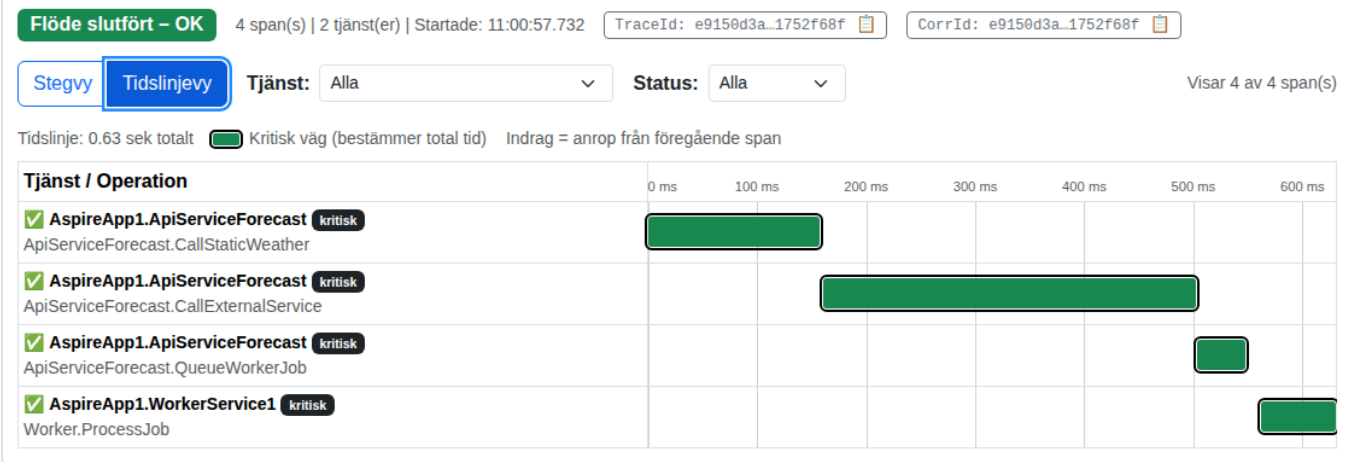
5. Ny sida: Insikter (/insights)

Valbart tidsfönster (1 h, 6 h, 24 h, 7 d), uppdateras var 10:e sekund. Diagrammen är inline-SVG utan JavaScript-beroenden. Statusfärger används alltid tillsammans med ikon och text.

#	Förslag	Vad det visar
1	Var fastnar flödena?	Vilket steg och vilken tjänst misslyckade körningar stannade på, med länk till senaste felet
2	Hälsa över tid	En rad per tjänst, sämsta status per intervall, upptid i %
3	Beroendediagram	Byggs från verklig trafik: antal anrop, felandel, snitttid
4	Trender	Klara, fel och pågående körningar per intervall
5	Svarstider	P50/P95/max och HTTP-fel per tjänst, topp 10 långsammaste operationer
6	Tidslinje i Processflöde	Tidsaxel, indrag efter förälder-barn, markerad kritisk väg
7	Retry, jobbkö, kedjor	Retry-frekvens per steg, jobbstatus per tjänst, kedjekörningar (tidigare osynliga)



Insikter med riktig data. WorkerService3 stängdes av för att framkalla fel, som syns som röd ruta och som stoppsteg.



Tidslinje: tidsaxel och kritisk väg (svart ram och etiketten "kritisk").

6. Kvalitet och verifiering

Kontroll	Resultat
Bygge (dotnet build)	0 varningar, 0 fel (varningar behandlas som fel i projektet)
Tester (dotnet run --project AspireApp1.Tests)	43/43 gröna: 20 befintliga, 3 SQLite-regression, 12 beräkningar, 6 tidslinje, 1 SQLite-integration, 1 statusberäkning
Körning i appen	Hela AppHost med SQLite, riktiga flöden, WorkerService3 avstängd. Playwright-skärmdumpar, inga konsolfel.
Arkitektur	Ren beräkningslogik (InsightsCalculator , GanttLayoutCalculator) skild från databas och UI. XML-kommentarer enligt CLAUDE.md.

7. Att känna till och nästa steg

- Den som vill använda SQL Server sätter StateStore__Provider=SqlServer .
- Insiktssidan läser upp till 20 000 rader per tabell var 10:e sekund. Det räcker för en demo, men vid större data bör aggregeringen flyttas till databasen eller cachas.
- Fler tjänster som skriver spans ger ett rikare beroendediagram.
- Startsidan visar en död tjänst som "Varning" (HTTP 0). Insiktssidan visar den korrekt som "Fel". Det kan justeras i en senare ändring.

Hälsningar från **Björn** och **Claude**. Vi byggde, felsökte och förbättrade AspireApp1 tillsammans.